

# Exercícios — Ponteiros, Structs e Arquivos

| Nível             | Quantidade              | O que cobra                                                                                                                        |
|-------------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Nível 0 — Fácil   | 30 exercícios (1 a 30)  | Um assunto por vez: ponteiros (1–10), structs (11–20) e arquivos (21–30)                                                           |
| Nível 1 — Médio   | 20 exercícios (31 a 50) | Assuntos combinados: aritmética de ponteiros, vetores de structs, structs com funções e -, leitura e escrita de arquivos com laços |
| Nível 2 — Difícil | 10 exercícios (51 a 60) | Sistemas completos que leem, processam e gravam arquivos usando structs e ponteiros                                                |

## Regras:

- Use **apenas** `#include <stdio.h>`.
- Sempre que o enunciado pedir uma função, crie a função e chame-a na main.
- Para strings, percorra o vetor de char até o `'\0'`, sem usar funções prontas.
- Todo `fopen` deve ser testado: se devolver `NULL`, mostre Erro ao abrir arquivo e encerre.
- Nos arquivos, nomes são sempre **uma palavra só** (sem espaços).

## Nível 0 — Fácil

### 1. Endereço e Valor

Declare `int x = 10;` e um ponteiro `p` apontando para `x`. Imprima `x` e `*p`. Depois, **usando só o ponteiro**, mude o valor para 20 e imprima `x` de novo.

#### Saída exemplo:

```
x = 10
*p = 10
x agora = 20
```

### 2. Dobrando pelo Ponteiro

Crie a função `void dobra(int *x)` que dobra o valor original. Leia um número, chame a função e imprima o resultado.

Entrada exemplo: 7

#### Saída exemplo:

```
Dobro: 14
```

### 3. Resultado por Ponteiro

Crie a função `void soma(int a, int b, int *resultado)`. A função não tem `return`: o resultado volta pelo ponteiro.

Entrada exemplo: 3 4

#### Saída exemplo:

```
Soma: 7
```

### 4. Troca

Crie a função `void troca(int *a, int *b)`. Leia dois números, troque-os e imprima.

Entrada exemplo: 3 7

#### Saída exemplo:

```
a = 7, b = 3
```

## 5. Percorrendo com Ponteiro

Leia 5 números para um vetor. Imprima-os usando `*(p + i)`, onde `p` aponta para o vetor (sem usar `v[i]` na impressão).

**Entrada exemplo:** 4 8 15 16 23

**Saída exemplo:**

```
4 8 15 16 23
```

## 6. Soma com p++

Leia 5 números para um vetor e some-os andando com um ponteiro (`p++`) do primeiro ao último elemento.

**Entrada exemplo:** 1 2 3 4 5

**Saída exemplo:**

```
Soma: 15
```

## 7. Maior e Menor de Dois

Crie a função `void ordena_dois(int a, int b, int *maior, int *menor)`.

**Entrada exemplo:** 8 3

**Saída exemplo:**

```
Maior: 8  
Menor: 3
```

## 8. Zerando o Vetor

Crie a função `void zera(int *v, int n)` que coloca 0 em todas as posições. Leia 4 números, mostre o vetor, chame a função e mostre de novo.

**Entrada exemplo:** 5 6 7 8

**Saída exemplo:**

```
Antes: 5 6 7 8  
Depois: 0 0 0 0
```

## 9. Tamanho com Ponteiro de char

Crie a função `int tamanho(char *s)` que conta os caracteres andando com o ponteiro até o `'\0'`.

**Entrada exemplo:** ponteiro

**Saída exemplo:**

```
Tamanho: 8
```

## 10. Divisão Segura

Crie a função `int divide(int a, int b, int *resultado)`. Se `b` for 0, ela retorna 0 e não mexe no resultado; senão, guarda `a / b` no ponteiro e retorna 1.

**Entrada exemplo:** 10 0

**Saída exemplo:**

```
Erro: divisao por zero
```

## 11. Ponto no Plano

Crie a struct `Ponto` com os campos inteiros `x` e `y`. Leia um ponto e imprima-o.

**Entrada exemplo:** 3 -2

**Saída exemplo:**

```
Ponto: (3, -2)
```

## 12. Ficha do Aluno

Crie a struct `Aluno` com nome (`char[50]`) e nota (`float`). Leia um aluno e mostre o nome e a situação (aprovado com nota  $\geq 7$ ).

**Entrada exemplo:** Ana 8.5

**Saída exemplo:**

```
Ana: Aprovado
```

## 13. Retângulo

Crie a struct `Retangulo` com base e altura (`float`). Leia um retângulo e mostre área e perímetro.

**Entrada exemplo:** 4 2.5

**Saída exemplo:**

```
Area: 10.000000
Perimetro: 13.000000
```

## 14. Ponto Médio

Usando uma struct `Ponto` com campos `float`, leia dois pontos e mostre o ponto médio entre eles.

**Entrada exemplo:** 1 2 5 8

**Saída exemplo:**

```
Ponto medio: (3.000000, 5.000000)
```

💡 O ponto médio é  $((x1 + x2) / 2, (y1 + y2) / 2)$ .

## 15. Data por Extenso

Crie a struct `Data` com dia, mes e ano. Leia uma data e mostre-a por extenso, usando `switch` para o nome do mês.

**Entrada exemplo:** 9 10 2026

**Saída exemplo:**

```
9 de outubro de 2026
```

## 16. Carrinho de Compras

Crie a struct `Produto` com nome e preço. Leia 3 produtos para um vetor de structs e mostre o total.

**Entrada exemplo:** Arroz 25.90 Feijao 8.50 Cafe 15.60

**Saída exemplo:**

```
Total: 50.000000
```

## 17. Struct como Parâmetro

Crie a struct `Boletim` com nome e três notas (`n1`, `n2`, `n3`). Crie a função `float media(Boletim b)`, que recebe a struct **por valor**.

**Entrada exemplo:** Bruno 6 7 8

**Saída exemplo:**

```
Bruno: 7.000000
```

## 18. Carro Antigo

Crie a struct `Carro` com modelo e ano. Leia um carro e diga se ele é antigo (fabricado antes de 2000).

**Entrada exemplo:** Fusca 1978

**Saída exemplo:**

```
Fusca e antigo
```

## 19. Aumento de Salário

Crie a struct `Funcionario` com nome e salario. Crie a função void `aumenta(Funcionario *f, float porcentagem)`, que altera o salário usando `->`.

**Entrada exemplo:** Carla 2000 10

**Saída exemplo:**

```
Carla: 2200.000000
```

## 20. Struct dentro de Struct

Crie a struct `Data` (dia, mês, ano) e a struct `Pessoa` com nome e nascimento (do tipo `Data`). Leia uma pessoa e mostre os dados.

**Entrada exemplo:** Diego 15 3 2004

**Saída exemplo:**

```
Diego nasceu em 15/3/2004
```

## 21. Primeiro Arquivo

Crie o arquivo `saida.txt` e escreva nele a frase `Ola, arquivo!`. Mostre na tela uma mensagem de confirmação.

**Saída exemplo:**

```
Arquivo criado
```

**Conteúdo de `saida.txt`:**

```
Ola, arquivo!
```

## 22. Lendo um Número

O arquivo `numero.txt` contém um número inteiro. Leia-o e mostre o dobro.

**Arquivo `numero.txt`:**

```
21
```

**Saída exemplo:**

```
Dobro: 42
```

## 23. Gravando de 1 a 10

Grave os números de 1 a 10 no arquivo `numeros.txt`, um por linha.

**Conteúdo de `numeros.txt`:**

```
1
2
...
10
```

## 24. Somando um Arquivo

Leia todos os números inteiros do arquivo `numeros.txt` (quantos houver) e mostre a soma.

**Arquivo `numeros.txt`:**

```
5
10
15
20
```

**Saída exemplo:**

```
Soma: 50
```

💡 `while (fscanf(arq, "%d", &x) == 1)` repete enquanto conseguir ler um número.

## 25. Contando Linhas

Conte quantas linhas tem o arquivo `texto.txt`, lendo caractere por caractere com `fgetc`.

**Arquivo `texto.txt`:**

```
primeira linha
segunda linha
terceira linha
```

**Saída exemplo:**

```
Linhas: 3
```

## 26. Copiando um Arquivo

Copie o conteúdo de `origem.txt` para  `copia.txt`, caractere por caractere, com `fgetc` e `fputc`.

**Arquivo `origem.txt`:**

```
Copiar e facil
com fgetc e fputc
```

**Saída exemplo:**

```
Copia concluida
```

**Conteúdo de `copia.txt`:** igual ao de `origem.txt`.

## 27. Diário de Execuções

Abra o arquivo `log.txt` no modo "a" e acrescente a linha `Programa executado`. Rodando o programa 3 vezes, o arquivo deve ter 3 linhas.

**Conteúdo de `log.txt` depois de 3 execuções:**

```
Programa executado
Programa executado
Programa executado
```

## 28. Salvando uma Pessoa

Leia nome e idade do teclado e grave no arquivo `pessoa.txt`, na mesma linha, separados por espaço.

**Entrada exemplo:** Ana 20

**Conteúdo de `pessoa.txt`:**

```
Ana 20
```

## 29. Arquivo que Não Existe

Tente abrir para leitura o arquivo `inexistente.txt` (que não existe). Mostre a mensagem de erro em vez de deixar o programa travar.

**Saída exemplo:**

```
Erro ao abrir arquivo
```

## 30. Contando Caracteres

Conte quantos caracteres (sem contar as quebras de linha) tem o arquivo `texto.txt`.

**Arquivo `texto.txt`:**

```
abc
de
```

**Saída exemplo:**

```
Caracteres: 5
```

## Nível 1 — Médio

### 31. Mínimo e Máximo por Ponteiro

Crie a função `void min_max(int *v, int n, int *min, int *max)`. Use aritmética de ponteiros (`*(v + i)`) para percorrer o vetor.

**Entrada exemplo:** `n=6 → 7 -3 12 0 5 9`

**Saída exemplo:**

```
Minimo: -3
Maximo: 12
```

### 32. Invertendo a Palavra

Crie a função `void inverte(char *s)` que inverte a palavra **nela mesma**, usando dois ponteiros: um no começo e outro no fim, trocando os caracteres e andando um em direção ao outro.

**Entrada exemplo:** ponteiro

**Saída exemplo:**

```
orietnop
```

### 33. Ponteiro para o Maior

Crie a função `int *maior(int *v, int n)` que **retorna um ponteiro** para o maior elemento. Na main, mostre o valor e a posição dele.

**Entrada exemplo:** `n=5 → 4 9 2 9 1`

**Saída exemplo:**

```
Maior: 9 na posicao 1
```

💡 Se `p` aponta para o maior, a posição é `p - v`. Em caso de empate, fique com o primeiro.

### 34. Vogais com Ponteiro

Crie a função `int conta_vogais(char *s)`, que percorre a palavra com um ponteiro (`s++`) e conta as vogais minúsculas.

**Entrada exemplo:** `programacao`

**Saída exemplo:**

```
Vogais: 5
```

### 35. Separando Pares e Ímpares

Crie a função `void separa(int *v, int n, int *pares, int *qp, int *impares, int *qi)`, que copia os pares para um vetor e os ímpares para outro, devolvendo as quantidades em `qp` e `qi`.

**Entrada exemplo:** `n=7 → 1 2 3 4 5 6 7`

**Saída exemplo:**

```
Pares: 2 4 6
Impares: 1 3 5 7
```

### 36. Melhor Aluno

Leia `n` alunos (nome e nota) para um vetor de structs e mostre o aluno com a maior nota.

**Entrada exemplo:** `n=3 → Ana 8.5 Bruno 9.2 Carla 7.0`

**Saída exemplo:**

```
Melhor: Bruno (9.200000)
```

### 37. Busca de Produto por Código

Crie a struct Produto com codigo, nome e preco. Leia n produtos e depois um código; crie a função int busca(Produto \*v, int n, int codigo) que retorna a posição do produto ou -1.

**Entrada exemplo:** n=3 → 10 Caneta 2.5 20 Lapis 1.0 30 Borracha 0.8, código 20

**Saída exemplo:**

```
Lapis: 1.000000
```

### 38. Conta Bancária

Crie a struct Conta com titular e saldo e as funções void deposita(Conta \*c, float valor) e int saca(Conta \*c, float valor) (retorna 0 se não houver saldo). Comece com saldo 0, deposite e saque os valores lidos.

**Entrada exemplo:** Ana, depósito 100, saque 150, saque 40

**Saída exemplo:**

```
Saldo insuficiente
Ana: 60.000000
```

### 39. Soma de Frações

Crie a struct Fracao (numerador e denominador), a função int mdc(int a, int b) e a função Fracao soma(Fracao a, Fracao b), que **retorna** a fração somada e simplificada.

**Entrada exemplo:** 1 4 1 6

**Saída exemplo:**

```
5/12
```

💡  $a/b + c/d = (a \cdot d + c \cdot b) / (b \cdot d)$ . Para simplificar, divida numerador e denominador pelo MDC deles.

### 40. Somando Tempos

Crie a struct Tempo (horas, minutos, segundos) e a função Tempo soma(Tempo a, Tempo b), que corrige o resultado para que minutos e segundos fiquem abaixo de 60.

**Entrada exemplo:** 1 45 50 2 30 20

**Saída exemplo:**

```
4 h 16 min 10 s
```

### 41. Comparando Datas

Crie a struct Data e a função int compara(Data a, Data b), que retorna -1 se a vem antes, 1 se vem depois e 0 se são iguais.

**Entrada exemplo:** 25 12 2025 9 10 2026

**Saída exemplo:**

```
A primeira data vem antes
```

### 42. Notas do Arquivo

O arquivo notas.txt tem, em cada linha, um nome e uma nota. Mostre a quantidade de alunos, a média e quantos foram aprovados (nota ≥ 7).

**Arquivo notas.txt:**

```
Ana 8.5
Bruno 6.0
Carla 9.0
Diego 5.5
```

**Saída exemplo:**

```
Alunos: 4
Media: 7.250000
Aprovados: 2
```

### 43. Tabuada em Arquivo

Leia um número e grave a tabuada dele (1 a 10) no arquivo `tabuada.txt`.

**Entrada exemplo:** 7

**Conteúdo de `tabuada.txt`:**

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

### 44. Separando em Dois Arquivos

Leia os números de `numeros.txt` e grave os pares em `pares.txt` e os ímpares em `impares.txt`. Mostre quantos foram para cada arquivo.

**Arquivo `numeros.txt`:**

```
3 8 12 7 5 10
```

**Saída exemplo:**

```
Pares: 3
Impares: 3
```

### 45. Contando Palavras

Conte quantas palavras tem o arquivo `texto.txt`, lendo uma palavra por vez com `fscanf(arq, "%s", palavra)`.

**Arquivo `texto.txt`:**

```
C e uma linguagem
muito usada ate hoje
```

**Saída exemplo:**

```
Palavras: 8
```

### 46. Maior Número e Sua Linha

O arquivo `valores.txt` tem um número por linha. Mostre o maior número e em qual linha ele está (a primeira linha é a 1).

**Arquivo `valores.txt`:**

```
15
42
-7
38
```

**Saída exemplo:**

```
Maior: 42 (linha 2)
```

### 47. Salvando Produtos

Leia  $n$  produtos (nome e preço) para um vetor de structs e grave-os em `produtos.txt`, um por linha, com o preço com duas casas decimais (`%.2f`).

**Entrada exemplo:**  $n=3 \rightarrow$  Caneta 2.5 Caderno 15 Mochila 89.9

**Conteúdo de `produtos.txt`:**

```
Caneta 2.50
Caderno 15.00
Mochila 89.90
```



`%.2f` mostra um float com exatamente 2 casas depois da vírgula.

## 48. Carregando Produtos

Leia o arquivo `produtos.txt` do exercício anterior para um vetor de structs. Mostre quantos produtos foram lidos, o mais caro e o valor total.

**Arquivo `produtos.txt`:**

```
Caneta 2.50
Caderno 15.00
Mochila 89.90
```

**Saída exemplo:**

```
Produtos: 3
Mais caro: Mochila
Total: 107.40
```

## 49. Contando uma Letra no Arquivo

Leia uma letra do teclado e conte quantas vezes ela aparece no arquivo `texto.txt`.

**Entrada exemplo:** a

**Arquivo `texto.txt`:**

```
banana amarela
abacate
```

**Saída exemplo:**

```
A letra a aparece 9 vezes
```

## 50. Arquivo em Maiúsculas

Leia `entrada.txt` caractere por caractere e grave em `saida.txt` o mesmo texto com todas as letras minúsculas convertidas para maiúsculas (os demais caracteres ficam iguais).

**Arquivo `entrada.txt`:**

```
Ola, mundo!
c em 2026
```

**Conteúdo de `saida.txt`:**

```
OLA, MUNDO!
C EM 2026
```



Só converta se o caractere estiver entre 'a' e 'z': `c = c - 'a' + 'A';`.

## Nível 2 — Difícil

### 51. Agenda de Contatos

Faça uma agenda com a struct `Contato` (nome e telefone, ambos `char[50]`) e um menu com `do-while` + `switch`: 1 = adicionar, 2 = listar, 3 = buscar pelo nome, 0 = sair. Ao **iniciar**, o programa carrega os contatos de `contatos.txt` (se o arquivo existir); ao **sair**, grava todos os contatos de volta no arquivo. Cada opção é uma função.

**Arquivo contatos.txt antes:**

```
Ana 81999990000
```

**Entrada exemplo:** 1 → Bruno 81988881111, 3 → Bruno, 2, 0

**Saída exemplo:**

```
Bruno: 81988881111
Ana 81999990000
Bruno 81988881111
```

**Conteúdo de contatos.txt depois:** os dois contatos, um por linha.

💡 Para buscar pelo nome, crie uma função `int iguais(char *a, char *b)` que compara as strings caractere por caractere.

### 52. Boletim da Turma

O arquivo `alunos.txt` tem, em cada linha, nome e três notas. Para cada aluno, calcule a média e grave em `boletim.txt` a linha `nome media situacao` (média com `%.2f`; situação `Aprovado` se média  $\geq 7$ ). No final, mostre na tela quantos foram aprovados e reprovados e quem teve a maior média.

**Arquivo alunos.txt:**

```
Ana 8 9 10
Bruno 5 6 4
Carla 7 7 8
```

**Saída exemplo:**

```
Aprovados: 2
Reprovados: 1
Maior media: Ana (9.00)
```

**Conteúdo de boletim.txt:**

```
Ana 9.00 Aprovado
Bruno 5.00 Reprovado
Carla 7.33 Aprovado
```

### 53. Ranking Ordenado

Leia o arquivo `alunos.txt` (nome e nota) para um vetor de structs, ordene-o da **maior para a menor** nota e grave o resultado em `ranking.txt`, com a posição na frente. Crie a função `void troca(Aluno *a, Aluno *b)` para trocar duas structs inteiras.

**Arquivo alunos.txt:**

```
Ana 7.5
Bruno 9.0
Carla 6.0
Diego 8.0
```

**Conteúdo de ranking.txt:**

```
1 Bruno 9.00
2 Diego 8.00
3 Ana 7.50
4 Carla 6.00
```

## 54. Controle de Estoque

O arquivo `estoque.txt` tem, por linha: código, nome, quantidade e preço. O arquivo `movimentos.txt` tem, por linha: código e uma quantidade (positiva = entrada, negativa = saída). Aplique os movimentos ao estoque (ignorando, com aviso, saídas maiores que a quantidade disponível), grave o resultado em `estoque_novo.txt` e mostre os produtos com quantidade abaixo de 5.

**Arquivo `estoque.txt`:**

```
1 Caneta 50 2.50
2 Caderno 10 15.00
3 Mochila 3 89.90
```

**Arquivo `movimentos.txt`:**

```
1 -20
2 -8
3 5
2 -5
```

**Saída exemplo:**

```
Movimento invalido: produto 2
Estoque baixo: Caderno (2)
```

**Conteúdo de `estoque_novo.txt`:**

```
1 Caneta 30 2.50
2 Caderno 2 15.00
3 Mochila 8 89.90
```

## 55. Juntando Arquivos Ordenados

Os arquivos `a.txt` e `b.txt` têm números inteiros em ordem crescente. Gere `c.txt` com todos os números em ordem crescente, lendo os dois arquivos **ao mesmo tempo**, sem guardar tudo num vetor e sem ordenar.

**Arquivo `a.txt`:** 1 4 9 12 **Arquivo `b.txt`:** 2 3 10 **Conteúdo de `c.txt`:**

```
1
2
3
4
9
10
12
```

💡 Mantenha o "próximo número" de cada arquivo numa variável. Grave o menor dos dois e leia o próximo só daquele arquivo. Quando um acabar, copie o resto do outro.

## 56. Estatísticas de Texto

Leia `texto.txt` com `fgetc` e mostre: número de linhas, de palavras, de letras e a letra mais frequente (considerando só letras minúsculas de a a z). Em caso de empate, mostre a que vem primeiro no alfabeto.

**Arquivo `texto.txt`:**

```
o rato roeu
a roupa do rei
```

**Saída exemplo:**

```
Linhas: 2
Palavras: 7
Letras: 20
Mais frequente: o
```

💡 Uma palavra começa quando você lê uma letra logo depois de um espaço, de uma quebra de linha ou do começo do arquivo. Para a letra mais frequente, use um vetor `int contagem[26]` e a posição `c - 'a'`.

## 57. Invertendo as Linhas

Leia as linhas de entrada.txt com fgets para um vetor de structs Linha (com um campo char texto[100]) e grave em saida.txt as linhas na ordem inversa (a última primeiro).

**Arquivo entrada.txt:**

```
primeira
segunda
terceira
```

**Conteúdo de saida.txt:**

```
terceira
segunda
primeira
```

💡 Cada linha lida pelo fgets já vem com o '\n' no final, então basta gravá-la com fprintf(arq, "%s", ...).

## 58. Banco com Extrato

Use a struct Conta (titular e saldo) com as funções deposita e saca recebendo Conta \*. A cada operação **bem-sucedida**, acrescente uma linha em extrato.txt (modo "a") no formato D valor ou S valor. Faça um menu (1 = depositar, 2 = sacar, 0 = sair). Ao sair, leia o extrato.txt e mostre todas as operações e o saldo final.

**Entrada exemplo:** Ana, depois 1 → 200, 2 → 50, 2 → 500, 1 → 30, 0 (com extrato.txt vazio no início)

**Saída exemplo:**

```
Saldo insuficiente
Deposito: 200.00
Saque: 50.00
Deposito: 30.00
Saldo final de Ana: 180.00
```

## 59. Cifra de César

Crie a função void cifra(char \*texto, int k), que desloca cada letra minúscula k posições no alfabeto (dando a volta de z para a) e não altera os outros caracteres. Leia k, cifre cada linha de mensagem.txt e grave em cifrado.txt. Depois, leia cifrado.txt, decifre (deslocando 26 - k) e mostre na tela, para provar que volta à mensagem original.

**Entrada exemplo:** 3

**Arquivo mensagem.txt:**

```
ataque ao amanhecer
xyz
```

**Conteúdo de cifrado.txt:**

```
dwdtxh dr dpdqkhfhu
abc
```

**Saída exemplo:**

```
ataque ao amanhecer
xyz
```

💡 Para dar a volta no alfabeto:  $c = (c - 'a' + k) \% 26 + 'a';$

## 60. Cadastro em Arquivo Binário

Leia  $n$  funcionários (struct com nome, idade e salario) e grave o vetor inteiro em `funcionarios.bin` com **um único** `fwrite`. Depois, abra o arquivo de novo, leia os registros com `fread` para outro vetor, mostre quantos foram lidos, liste-os e mostre a média salarial.

**Entrada exemplo:**  $n=3 \rightarrow$  Ana 30 3500 Bruno 25 2800 Carla 41 5200

**Saída exemplo:**

```
Registros lidos: 3
Ana 30 3500.00
Bruno 25 2800.00
Carla 41 5200.00
Media salarial: 3833.33
```

💡 `fwrite(vetor, sizeof(Funcionario), n, arq)` grava  $n$  structs de uma vez. Para ler sem saber quantos são, peça mais do que precisa: `fread(lidos, sizeof(Funcionario), 100, arq)` devolve quantos realmente leu.